Fire Assignment 12

by Lorraine Gaudio

Lesson generated on November 16, 2025

BOISE STATE UNIVERSITY

Contents

1. ☒ Point-Boost Activity

 Idaho is well known for its wildfires. They play a ecological role, shaping our forests and grasslands while presenting significant hazards to communities and resources. In recent years, the Idaho landscape has experienced frequent and sometimes severe wildfire seasons due to both natural and human activity. While fires are part of the natural ecosystem, they have become more destructive as the climate warms. Wildfire preparedness and management is a continuing challenge for the state. For more information this research at Boise State.

How far could a new wildfire run in 8 hours? How many new wild fire ignitions will grow larger than 10 km (straight line) in 8 hours?

This is an optional two-part point-boost activity. You can work in groups but each person submits their own work independently. Please record group members within your document.

Lesson Overview

By the end of this Point Boost Fire Activity you will be able to:

1. Organize – Set up a reproducible RStudio workflow (project, files, data, RData) that can be re-run on another machine.
2. Wrangle – Import the WHP sample data, create quantile-based hazard categories, and summarize areas and probabilities with tidy R code.
3. Model – Specify and use lognormal models (p/q/d/r *lnorm* functions) to answer distance and tail-probability questions by hazard category.
4. Simulate – Build a multi-step Monte Carlo simulation (with replicate()) that mixes categories by area weights and estimates the chance of many fast-growing fires in a season.
5. Communicate – Knit a clear Rmd/Qmd notebook that explains assumptions, code choices, and results, with appropriate metadata and citations for data and help sources.
6. Reflect – Document troubleshooting, AI use, and learning-skill growth (curiosity, courage, tenacity, thoroughness, open-mindedness) through short memos embedded in the workflow.

1.1 Assignment Directions

Read the background material below, then complete the tasks that follow. Replace each ____ placeholder (and any TODO comments) with working code or a short written answer. Run each section; be sure the requested objects appear in the Environment.

1. **Technical Skill Points**

You could earn *up to* **10** points in Technical skill. This assignment will require code covered in modules 1 through 13. You will load a CSV file, manipulate the data frame with dplyr verbs, simulate random numbers and visualize results. Additionally, you will be asked to use functions that were not covered in any modules. You will need to look up the function documentation and figure out how to use them on your own. The submission must include an Rmd (or Qmd), HTML report, and RData of the environment.

2. **Learning Skill Points**

You could earn up to **10** points Learning skill on this optional Point Boost assignment!

Gate: R code with memos submitted.

Critical Checkpoint: R code with memos submitted that meets all criteria below. ALL must be YES or the score is 0 (per section/part).

Use the template □ Goal → □ Code → □ Checks → □ Comment to met the Critical Checkpoint.

1. □ **Goal** – In your own words, one sentence about what you are trying to do.
2. □ **Code** – Code that completes the task (replace each ____ with your own code).
3. □ **Checks** – Code that convinces you your solution works (structure, counts, summaries, etc.).
4. □ **Comment** – 1–2 sentences either:

- data-focused (what you learned about the wildfire data), **or**
- code-focused (what the code does / why it works).

Supporting

- □ Format your document with headings, code chunks, and narrative text. Use proper grammar, spelling, and punctuation. Include a document title, author name, and □ date in the heading of your report.
- □ Cite resources (including referencing lessons, peers, instructor guidance, internet websites, AI summaries, the help file in R) in memos. State specifically what or who helped and in what ways. Always compose your own code (no copy and paste).
- □ Code trials show how you explored ideas, asked questions independently of the assignment. Include code comments to explain your thinking.
- □ Reflect on learning process, connects ideas, discusses challenges, and describes improvement strategies.
- □ Growth mindset means you demonstrate effort, persistence, and a positive attitude toward learning new skills. You show courage.
- □ Prediction means you make hypotheses about what the code will do or what the data will look like before running it, and then reflect on whether your predictions were correct.
- □ Record what you tried, even if it didn't work. This is proof of thoroughness and trustworthiness, prevents future duplication of errors, contextualizing successes, and exposes hidden discoveries (unanticipated effect).

3. **Signature Assignment Option**

You may choose to direct these points to your Signature Assignment requirement. If you do, you **must** complete all tasks and submit a polished HTML document with clear code, thorough comments, and well-written explanations.

When finished, □ Knit or Render your report to HTML, and upload with your RData to Canvas by or before December 12th.

1.2 Dataset

Wildfire managers need tools that show where fires are more likely and how intense they could become. The U.S. Forest Service's Wildfire Hazard Potential (WHP) is one such tool: a nationwide raster layer where each grid cell stores a unitless index of relative wildfire hazard (higher value = higher hazard). For this activity we use an Idaho-only WHP file prepared by Dr. Matt Williamson SPASES Lab Page.

Figure 1 shows the Idaho WHP raster created in R with the packages terra (raster operations), sf (vector data), tidyverse (data wrangling), tidyterra (ggplot helpers for rasters), and tigris (state/county boundaries). The GeoTIFF uses an Albers Equal Area projection (units: meters) with 240 × 240 m cells. Within Idaho, valid cells cover about 216,315 km²; cells outside the state within the file's bounding box are NoData. WHP values in this subset range from 0 to 64,185.656; the median is about 397, and the upper tail is steep (e.g., 95th percentile ≈ 7,753, 99th ≈ 22,429).

Formal dataset citation

Dillon, Gregory K.; Gilbertson-Day, Julie W. 2020. Wildfire Hazard Potential for the United States (270-m), version 2020. 3rd Edition. Forest Service Research Data Archive. https://doi.org/10.2737/RDS-2015-0047-3.

Idaho subset (whp_id.tif) prepared and provided by Matt Williamson, SPASES Lab, Boise State University.

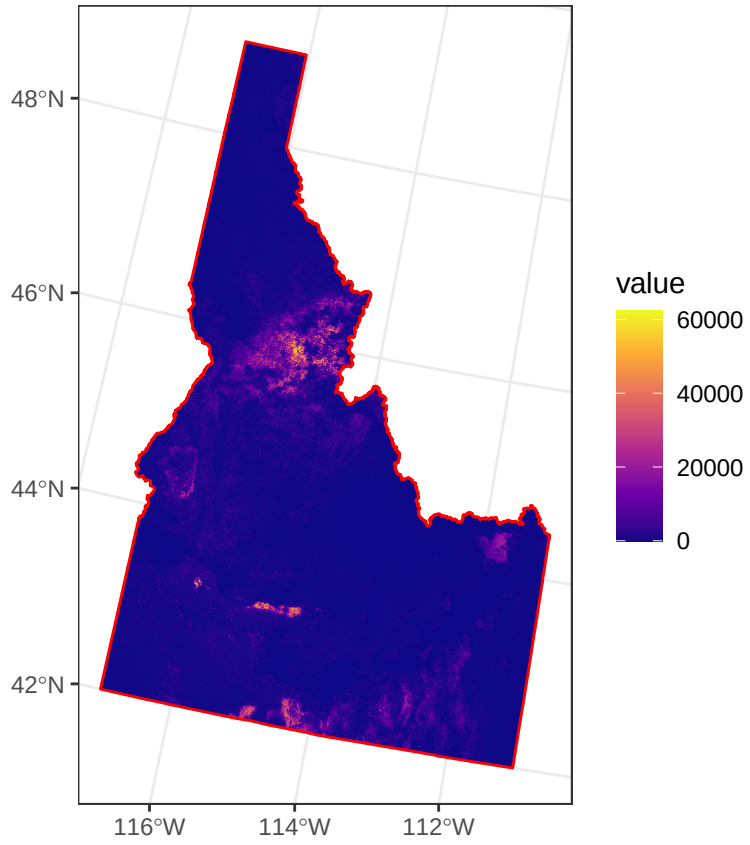

Figure 1.1: Figure 1. Idaho Wildfire Hazard Potential (WHP), continuous index (unitless) from the USFS WHP v2020 raster, clipped to the Idaho boundary. Colors use a viridis gradient (dark purple = lower WHP; yellow = higher WHP); values span approximately 0–64,186. Raster resolution is 240 m in an Albers Equal Area projection; NoData cells are transparent. Red outline shows the TIGER/Line state boundary reprojected to the raster CRS. Idaho subset prepared by Matt Williamson (SPASES Lab, Boise State University); original dataset: Dillon & Gilbertson-Day (2020), USFS Research Data Archive (DOI: 10.2737/RDS-2015-0047-3).

2. Part A

Explore and wrangling of the raw WHP sample data.

2.1 Task 1

Load Data

Task instructions:

1. Use **readr** to read the file whp_raw_sample.csv from your working directory.
2. Store the result in an object named **county_raw** in your Environment.
3. Run at least two **structural checks** to make sure the data imported correctly (for example: number of rows, variable names, a summary of key columns).
4. Write a short comment describing what kind of dataset county_raw is (size, what a row represents, high-level purpose).

 Goal.
Write your own one-sentence goal here (e.g., what you want to load and understand).

```
#  Code: readr::read_csv
_____ # 1. e.g., load any needed package(s)
_____ # 2. e.g., read the CSV into county_raw
```

```
#  Explore Data
_____ # 3. e.g., structure or glimpse of county_raw
_____ # e.g., check column names or a summary of whp / cell_area_km2
```

 Comment (TODO). *In 1–2 sentences, describe what county_raw represents (rows, key columns, why it might be useful).*

2.1.1 Column meanings

whp – The Wildfire Hazard Potential value for a single sampled raster cell. This is a unitless, continuous index (higher = greater relative hazard). Values come directly from the WHP GeoTIFF.

county – The county name (e.g., "Idaho", "Twin Falls") obtained by spatially joining each sampled cell to the TIGER/Line county boundary. Use for readable reports.

county_fips – The county's 3-digit FIPS code from TIGER/Line. *Option: To create the standard 5-digit FIPS for Idaho, prepend the state code 16: e.g., 049 → 16049. Need to use the 5-digit code when joining to external datasets (ACS, MTBS summaries, etc.).*

cell_area_km2 – The area (in square kilometers) represented by that raster cell, computed from the raster's equal-area projection. With 240 m × 240 m cells this is ~0.0576 km²; small numeric differences reflect rounding. Summing this column by any grouping yields true area totals, and dividing by the statewide sum yields area shares.

2.2 Task 2

Basic data audit summaries

Task instructions:

1. Using dplyr::summarize() (or base R equivalents), compute and report the following for county_raw:

- Total number of rows.
- Number of missing values in whp.
- Minimum value of cell_area_km2.
- Maximum value of cell_area_km2.

2. Store the summary tibble (or data frame) in a clearly named object (e.g., county_summary).

□ Goal. *Write your own one-sentence goal about what you are summarizing.*

```
# □ Code: summarize key audit quantities for county_raw
_____ # 1/2. assign county_summary and use county_raw
_____ # 1. compute n_rows
_____ # 1. compute n_na_whp
_____ # 1. compute min_area
_____ # 1. compute max_area
```

□ Checks: print the summary object and confirm the summary makes sense.

□ Comment (TODO). *In 1–2 sentences, interpret what these numbers say about the dataset quality or structure (missingness, range of area, etc.).*

2.3 Task 3

Compute WHP quantile edges (category cut points)

Task instructions

1. Use stats::quantile() on the whp column of county_raw to compute the quantiles for prob using 0, 0.25, 0.50, 0.75, 0.90, 0.95, 0.99, 1.00.
2. Store the result in an object named e (for "edges").
3. Usena.rm = TRUE so missing values are ignored.
4. Look at e and make sure it has 8 ordered values that look reasonable for a right-skewed variable.

□ The SYNTAX: quantile(x, probs, na.rm)

□ Goal. *Write your own goal explaining quantiles/edges goal.*

```
# □ Code: Base R stats::quantile
____ # 2. store WHP quantile edges in object e
____ # 1. call quantile() on county_raw's whp column
____ # 1. specify the probs vector
____ # 3. include remove NA from calculation
```

```
# □ Verify
____ # print e
```

□ Comment (TODO). *In 1–2 sentences, describe what e represents and how these cut points will be used in later tasks.*

Task 3 might have been challenging! Consider other kinds of memos: □ Reflect; □ Growth mindset; □ Cite resources; □ Code trials; □ Record what you tried.

□ Verify Option: Each element corresponds to a specific percentile:

- e[1] = minimum WHP (0th percentile)
- e[2] = 25th percentile
- e[3] = 50th percentile (median)
- e[4] = 75th percentile
- e[5] = 90th percentile
- e[6] = 95th percentile
- e[7] = 99th percentile
- e[8] = maximum WHP (100th percentile)

2.4 Task 4

Add a categorical WHP risk label

Task instructions:

1. Start from county_raw, create a new data frame called county_cats.
2. Use dplyr::mutate() and dplyr::case_when() to add a new variable Category that assign a label based on whp and the edges in e.
3. Use the following intervals (closed on the left (>=), open on the right (<), except the last, which is closed on both sides (>=; <=)):

- If a row's whp value is *at least* the minimum WHP AND *less than* the 25th percentile, label it "Very Low".
- If whp is *at least* the 25th percentile AND *less than* the 50th percentile (median), label it "Low".
- If whp is *at least* the 50th percentile AND *less than* the 75th percentile, label it "Moderate".
- If whp is *at least* the 75th percentile AND *less than* the 90th percentile, label it "High".
- If whp is *at least* the 90th percentile AND *less than* the 95th percentile, label it "Very High".
- If whp is *at least* the 95th percentile AND *less than* the 99th percentile, label it "Extreme".
- If whp is *at least* the 99th percentile AND *less than or equal to* the maximum WHP, label it "Ultra (Top 1%)".
- If a value of whp does not fall in any of these ranges (e.g., NA), Category should be NA.

4. Convert Category to an **ordered factor** so the levels follow the intended risk order.

- "Very Low", "Low", "Moderate", "High", "Very High", "Extreme", "Ultra (Top 1%)".

□ Goal. *Write your own sentence(s) goal for turning numeric WHP into ordered risk categories.*

```
# □ Code: dplyr::mutate + dplyr::case_when
_____ # 1. assign county_cats and pipe county_raw into it
_____ # 2. use mutate() to create Category
_____ # 3. with case_when() and e[ ] bounds
_____ # 4. convert Category to an ordered factor with the specified levels
```

```
# □ Verify: confirm Category looks right
_____ # e.g., count() or table()
_____ # optional: str() or glimpse()
```

□ Comment (TODO). *Either: (a) explain briefly how the case_when logic translates WHP into categories, or (b) interpret what the distribution of categories says about Idaho's wildfire hazard.*

Task 4 might have been challenging! Consider other kinds of memos: □ Reflect; □ Growth mindset; □ Cite resources; □ Code trials; □ Record what you tried.

2.5 Task 5

Aggregate area and compute category probabilities

1. Start from county_cats
2. Use dplyr::group_by() with Category.
3. Use summarize() to compute, for each Category:
   - area_km2 equals the sum of cell_area_km2.
   - n_cells equals the number of grid cells using n().
   - Use .groups = "drop" so that the result is ungrouped.
4. Use dplyr::mutate() to create a new variable called Probability that is the area_km2 divided by sum(area_km2).
5. Use dplyr::select() to keep only Category, Probability, area_km2, n_cells in this order.

□ Goal. *Write your own one-sentence goal for summarizing WHP categories into areas and probabilities.*

□ Prediction.

```
# □ Code: dplyr::group_by + dplyr::summarize + dplyr::mutate
_____ # 1. store prob_tbl and pipe county_cats into it
_____ # 2. group_by()
_____ # 3. summarize()
_____ # 4. mutate())
_____ # 5. select()
```

```
# □ Verify: validate prob_tbl
_____ # print prob_tbl
_____ # check that probabilities sum to ~1
```

□ Comment (TODO). *In 1–2 sentences, describe what prob_tbl tells you about the share of Idaho's area in each hazard category and how that connects to "probability of a random ignition's category."*

3. Part B

The table below summarizes the quantile-based wildfire hazard categories derived from Idaho's continuous USFS WHP raster and the distance distributions used for an 8-hour run. The **Category** column lists seven bins (Very Low to Ultra) defined from the raster's cumulative distribution; the WHP index itself is unitless. The **Probability** column is the statewide area share for each bin—expressed as a percent of valid raster area (NA excluded)—and is interpreted as the chance that a random ignition falls in that category.

Category	Probability	Mean (km)	SD (km)	Mean log	SD log
Very Low	25	2.0	1.0	0.582	0.472
Low	25	3.0	1.0	1.046	0.325
Moderate	25	4.5	1.5	1.451	0.325
High	15	7.0	2.0	1.907	0.280
Very High	5	12.0	3.0	2.455	0.246
Extreme	4	18.0	4.0	2.866	0.220
Ultra (Top 1%)	1	28.0	6.0	3.310	0.212

The table also defines a simple fire-spread scenario using "plausible values" derived from ChatGPT for an 8-hour straight-line distance by hazard category. These distances are not estimated from the WHP raster. They are modeling assumptions chosen for this assignment. Specifically, we assume that (1) ignitions are equally likely anywhere within the valid WHP raster (so Probability reflects area share), (2) higher WHP categories correspond to larger 8-hour straight-line spread distances, and (3) within each category, 8-hour distances follow either a zero-truncated normal distribution with the given mean and SD or a lognormal distribution.

If you are looking at the PDF, you will see mathematical notation. **You do not need to understand these equations to complete the tasks.** There are there for those who are studying statistics more deeply (e.g., your instructor). I've placed them in custom text boxes to set them apart. If you are looking at the HTML version, the equations will not be shown.

The Normal mean (km) and Normal SD (km) columns give the mean and standard deviation, in kilometers, for a zero-truncated normal distance model representing straight-line spread over 8 hours. The Mean log and SD log columns give the parameters μ and σ of a lognormal distance model in which

$$\log(\text{distance in km}) \sim \mathcal{N}(\mu, \sigma^2).$$

Labels "Extreme" and "Ultra" denote upper-tail bins (approximately the 95–99

3.1 Statistical Methods

A continuous random variable is a variable that can take on an infinite number of values within a given range. Unlike discrete random variables, which can only take on specific, separate values (like the roll of a die), continuous random variables can take on any value within a certain interval. Examples of continuous random variables include weight or time. In this assignment, we will use distance in km.

> Let X be the continuous random variable of straight-line kilometer distance in an 8-hour period. Then a probability distribution of X is a function $f(x)$ such that for any two numbers a and b with $a \leq b$,
>
> $$P(a \leq X \leq b) = \int_a^b f(x)\,dx$$
>
> The probability that X takes on a value in the interval $[a, b]$ is given by the area above this interval and under the graph of the density function.

3.2 Lognormal Distribution

In this assignment, we will use **lognormal distribution** to model the 8-hour straight-line distance of new ignitions in High wildfire hazard categories. We will explore the list of R functions for the lognormal distribution. You can access the documentation for these functions by running the following command in R:

```
?Lognormal
```

3.3 Task 1

Tail probability with plnorm()

plnorm() computes the cumulative distribution function (CDF) for a lognormal distribution. It gives the probability that a lognormally distributed random variable is less than or equal to a certain value.

 Question a: If a new ignition starts in a **High-hazard area**, what is the probability that its 8-hour straight-line spread is at least 10 km?

Let X be the continuous random variable giving the 8-hour straight-line spread distance (in km) of a new ignition in a High-hazard area. Suppose X has the probability density function $f_{high}(x)$. Then for any two distances a and b with $0 \le a \le b$,

$$P(a \le X \le b) = \int_a^b f_{high}(x)dx.$$

In particular, the probability that the 8-hour straight-line spread is at least 10 km is given by

$$P(X \le 10) = \int_{10}^{\infty} f_{high}(x)dx,$$

and in this problem, $f_{high}(x)$ is the density of a lognormal distribution.

Task instructions

1. Treat X as the 8-hour distance (km) for a new ignition in a High-hazard area.
2. Model X with a lognormal distribution using the **High** row's **Mean log** and **SD log** from **Table 1**.
3. Use plnorm() with lower.tail = FALSE to compute $P(X \ge 10) = P(X > 10)$.
4. Store the result as **p_high_exact**.
5. Check both the raw probability and the corresponding percentage.

□ **Goal.**
In your own words, state what you are finding the probability of, including time and distance and model type.

```
# □ Test: lognormal tail probability for High category
_____ # store p_high_exact, use plnorm(), 10 km threshold, High Mean log, High SD log
```

```
# □ Verify: eexamine probability and percentage
_____ # display p_high_exact
_____ # convert p_high_exact to a percentage
```

□ Comment (TODO). *Interpret what your probability means for High-hazard ignitions.*

3.4 Task 2

qlnorm() computes the quantile function (inverse CDF) for a lognormal distribution. It gives the value below which a certain percentage of the distribution falls.

□ Question b: We'll treat distances above the 95th percentile as 'extreme fast runs'. For ignitions in High-hazard areas, what 8-hour distance is only exceeded 10% of the time? In other words, what is the 95th percentile run length?

Let X be the continuous random variable giving the 8-hour straight-line spread distance (in km) of a new ignition in a High-hazard area. Suppose X has the probability density function $f_{high}(x)$. The cumulative distribution function (CDF) of X is given by

$$F(x) = P(X \leq x) = \int_0^x f_{high}(t)dt.$$

We define the 95th percentile of X, denoted by $q_{0.95}$ of this distribution to be the value satisfying

$$P(X \leq q_{0.95}) = F(q_{0.95}) = 0.95.$$

equivalently,

$$q_{0.95} = \int_0^{q_{0.95}} f_{high}(x)dx.$$

In particular, the 95th percentile of the 8-hour straight-line spread distance for new ignitions in High-hazard areas is given by

$$q_{0.95} = F^{-1}(0.95),$$

and in this problem $f_{high}(x)$ is the density of a lognormal distribution.

Task instructions

1. Continue modeling the High-hazard 8-hour distance X with a lognormal distribution.
2. Use qlnorm() to find the 95th percentile distance (km) for X.
3. Use the High row's Mean log and SD log from Table 1.
4. Store the result as q95_high.
5. Check that this distance is plausible given the High category's mean and SD.

□ Goal. *Write your own sentence(s) goal for finding a distance that 95% of High-hazard ignitions stay below.*

```
# □ Test: 95th percentile distance for High category
____ # store q95_high, use qlnorm() with p = 0.95, High Mean log, High SD log
```

```
# □ Verify: print and sanity-check the 95th percentile
____ # display q95_high
```

□ Comment (TODO). *Explain in words what this percentile distance means in the context of High-hazard ignitions.*

3.5 Task 3

dlnorm() computes the probability density function (PDF) for a lognormal distribution. It gives the relative likelihood of the random variable taking on a specific value.

 Question c: Under our High-hazard lognormal model, how concentrated the distribution is locally around the shorter 3 km run verses the longer 10 km distance in 8 hours? *Note: Higher density indicates a more likely value in a small neighborhood around that point.*

Let X be the continuous random variable giving the 8-hour straight-line spread distance (in km) of a new ignition in a High-hazard area. Suppose X has the probability density function $f_{high}(x)$. Then for any two distances a and b with $0 \leq a \leq b$,

$$P(a \leq X \leq b) = \int_a^b f_{high}(x)dx.$$

The values $f_{high}(x)$ are densities, not probabilities. They represent the height of the distribution at the point x. In particular, the density at 3 km and 10 km are given by

$$f_{high}(3) \text{ and } f_{high}(10).$$

Task instructions

1. Under the High-hazard lognormal model, use dlnorm() to compute the density at 3 km and 10 km.
2. Use the High row's Mean log and SD log from Table 1.
3. Store the results as d_3 and d_10.
4. Compare d_3 and d_10 to decide which distance is more locally likely.

 Goal. *Write your own sentence(s) goal for comparing how locally likely 3 km vs 10 km runs are under the High-hazard model.*

```
#  Test: compare densities at 3 km and 10 km
_____ # compute d_3 using dlnorm() at 3 km with High Mean log, High SD log
_____ # compute d_10 using dlnorm() at 10 km with High Mean log, High SD log
```

```
#  Checks: print and compare heights

_____ # display d_3
_____ # display d_10
```

 Comment (TODO). *Use your two density values to describe which distance is more locally likely and what that implies.*

3.6 Task 4

rlnorm() generates random numbers from a lognormal distribution. **Note**: This task has three parts.

 Question d: If we simulate 450 new ignitions in High-hazard areas this season, about how many would we expect to classify as 'fast-growing' (≥10 km in 8 hours)?

Let X be the continuous random variable giving the 8-hour straight-line spread distance (in km) of a new ignition in a High-hazard area. Assume

$$X \sim \text{Lognormal}(\mu = \text{meanlog High}, \sigma = \text{ meanlog High}),$$

with density function $f_{high}(x)$ and CDF $F_{high}(x)$.

The probability that a single new ignition is 'fast-growing' (i.e., spreads 10 km or more in 8 hours) is given by

$$P(X \geq 10) = 1 - F_{high}(10)$$

Now suppose we observe (or simulate) $n = 450$ new ignitions in High-hazard areas this season. Let $X_1, X_2, \ldots, X_{450}$ be random variables, each with the same lognormal distribution as X.

Define indicator variables for 'fast-growing' ignitions:

$$I_i = \begin{cases} 1 & \text{if } X_i \geq 10 \\ 0 & \text{if } X_i < 10 \end{cases} \quad i = 1, \ldots, 450.$$

Then, the total number of 'fast-growing' ignitions, S, is given by

$$S = \sum_{i=1}^{450} I_i$$

Each I_i is a Bernoulli random variable with success probability $P(I_i = 1) = P(X_i \geq 10) = P(X \geq 10)$, so, the expected number of 'fast-growing' ignitions is

$$\mathbb{E}[S] = \sum_{i=1}^{450} \mathbb{E}[I_i] = 450 \cdot P(X \geq 10).$$

3.6.1 Task 4.1

Simulate High-hazard distances

Task instructions

1. Use rlnorm() to simulate 450 independent 8-hour distances for ignitions in the High category.

2. Use the High row's Mean log and SD log from Table 1.
3. Set a seed with set.seed(2025) for reproducibility.
4. Store the simulated distances in a vector High_Lognorm.
5. Summarize the simulated distances to understand their range and typical values.

□ Goal. *Write your own sentence(s) goal for Task 4.1.*

```
# □ Test: simulate 8-hour distances for High category
_____ # set a seed for reproducibility
_____ # store High_Lognorm, use rlnorm() to generate 450 values with High Mean log, High SD log
```

```
# □ Verify: look at basic summaries
_____ # summarize High_Lognorm (e.g., summary())
_____ # confirm the length of High_Lognorm is 450)
```

□ Comment (TODO). *Describe briefly how the simulated distances behave (range, skew, center).*

3.6.2 Task 4.2

Count fast-growing ignitions (≥10 km)

Task instructions:

1. Define a "fast-growing" ignition as one with an 8-hour distance ≥ 10 km.
2. Use a logical condition on High_Lognorm to identify fast-growing runs.
3. Use sum() on that logical vector to count how many simulated ignitions are fast-growing.
4. Store the count as fast_growing_count.
5. Optionally, compute the simulated proportion of fast-growing ignitions and compare it to p_high_exact from Task 1.

□ Goal. *Write your own sentence(s) goal for Task 4.2.*

```
# □ Test: count fast-growing ignitions (≥ 10 km)
_____ # store as fast_growing_count,use sum() to count TRUEs of logical vector
```

```
# □ Verify: inspect count (and optional proportion)

_____ # display fast_growing_count
_____ # optionally compute the proportion fast_growing_count / length(High_Lognorm)
```

□ Comment (TODO). *Explain what your simulated count suggests about how many fast-growing events you might see in a season.*

3.6.3 Task 4.3

Visualize simulated distances

Task instructions:

1. Use hist() to create a histogram of High_Lognorm.
2. Choose a reasonable bin setting and axis labels.
3. Use the plot to comment on the distribution shape (e.g., skew, tail length).
4. Connect the shape of the histogram to your numeric summaries from Task 4.1.

□ Goal. *Write your own sentence(s) goal for visualizing the simulated High-hazard distances.*

```
# □ /□ Code histogram of simulated High_Lognorm distances
____ # call hist() on High_Lognorm, choosing breaks/title/xlab as you see fit
```

□ Comment (TODO). *In one sentence, relate the histogram's shape to your earlier summaries and to what you expect from a lognormal model.*

4. Part C

rlnorm + replicate()

 Question e: In a season with 450 new ignitions **statewide**, what is the probability we see at least 50 fast-growing events (≥10 km in 8 hours) when ignitions can occur in any hazard category according to their statewide area weights?

Let there be L hazard categories indexed by $i = 1, 2, \ldots, L$ with:

- statewide area weights p_j such that $\sum_{j=1}^{L} p_j = 1$.

- lognormal spread model in category j:

$X|\text{category} = j \sim \text{Lognormal}(\text{meanlog}_j, \text{sdlog}_j)$.

Consider a season with $N = 450$ new ignitions **statewide**. For each ignition $i = 1, 2, \ldots, N$:

1. Draw a hazard category

$$C_i \in \{1, 2, \ldots, L\}, P(C_i = j) = p_j.$$

independently across ignitions.

2. Conditional on $C_i = j$, draw an 8-hour straight-line spread distance X_i as

$$X_i|C_i = j \sim \text{Lognormal}(\text{meanlog}_j, \text{sdlog}_j).$$

independently across ignition given their categories.

Define the indicator variable for a 'fast-growing' ignition ($\geq k = 10$ km in 8 hours) as

$$I_i = \begin{cases} 1 & \text{if } X_i \geq k \\ 0 & \text{if } X_i < k \end{cases} \quad i = 1, \ldots, 450.$$

Then, the total number of 'fast-growing' ignitions in the season, S, is given by

$$T = \sum_{i=1}^{450} I_i$$

We are interested in the probability that a season has at least 5 fast-growing events:

$$P(T \geq 50).$$

and is approximated by Monte Carlo simulation.

$$\widehat{P(T \geq 50)} \approx \frac{1}{S} \sum_{s=1}^{S} \mathbf{1}\{T^{(s)} \geq 50\},$$

where $T^{(s)}$ is the simulated count of fast-growing ignitions in season s, and S is the total number of simulated seasons.

4.1 Task 1

Create category labels Task instructions

1. Using **Table 1**, create a character vector named **labels** that contains the seven wildfire hazard categories in the same order as the **Category** column.
2. Make sure the spelling and capitalization of each label exactly matches Table 1.
3. Check that the vector prints correctly.

□ **Goal.**
Write your own sentence(s) goal for creating a vector of category labels.

```
# □ Test: vector of category labels
____ # create vector <- c( ... ) using the Category names from Table 1
```

```
# □ Verify: inspect labels

____ # display labels
```

□ Comment (TODO). *Explain briefly why matching the order and spelling of these labels will matter later.*

4.2 Task 2

Create category probabilities

Task instructions

1. Using the Probability column in Table 1 (as proportions that sum to 1), create a numeric vector **prob**.
2. The entries of prob must be in the same order as labels.
3. Check that the probabilities sum to 1 (within rounding error).

□ Goal. *Write your own sentence(s) goal for building vector.*

```
# □ Test: vector of category probabilities
____ # create vector <- c( ... ) using the Probability column from Table 1
```

```
# □ Verify: : confirm they form a valid distribution
____ # display vector
____ # check that sum() is 1
```

4.3 Task 3

Create lognormal parameter vectors

Task instructions

1. From Table 1, read off the Mean log values for each category in the same order as labels and store them in a vector **meanlog**.
2. Similarly, read off the SD log values and store them in a vector **sdlog**.
3. Check that both vectors line up with **labels** (same length, same order).

□ Goal. *Write your own sentence(s) goal for vectors.*

```
# □ Test: vectors of lognormal parameters
____ # create vector 1 <- c( ...) from the Mean log column
____ # create vector 2 <- c( ...) from the SD log column
```

```
# □ Verify: inspect and compare
____ # display vector 1
____ # display vector 2
____ # check length of labels vector, with vector 1 and 2.
```

□ Comment (TODO). *Describe any pattern you see*

4.4 Task 4

Set global simulation constants

Task instructions

1. Let **N** be the number of ignitions per season (use the value given in the problem statement).
2. Choose a large number **S** of simulated seasons (as specified in the assignment).
3. Set the "fast-growing" threshold **k** in km (use the value given in the problem statement).
4. Store these three values in objects N, S, and k.

□ Goal. *Write your own sentence(s) goal for setting up the core constants.*

```
# □ Test: number of ignitions, seasons, and fast threshold
____ # set N to the number of ignitions per season
____ # set S to the number of simulated seasons
____ # set k to the fast-run distance threshold in km
```

```
# □ Verify: inspect constants
____ # display N
____ # display S
____ # display k
```

4.5 Task 5

Sample categories for one season

Task instructions

1. Simulate one season of ignitions by drawing N categories according to their probabilities.
2. Use sample.int() (or sample()) to draw indices from 1:length(labels) with replacement and probability vector prob.
3. Store the category indices in a vector named **idx**.
4. Use table() with labels[idx] to see how many ignitions fall into each category.

□ Goal. *Write your own sentence(s) goal for simulating a single season's mix of categories.*

```
# □ Test: sample category indices for one season
____ # create idx using sample.int() (or sample()) with size = N, prob = prob
```

```
# □ Verify: examine realized category mix

____ # inspect the first few indices
____ # use table() to see counts per category
```

□ Comment (TODO). *Compare your simulated category counts to the theoretical proportions from prob.*

4.6 Task 6

Draw distances for one season

Task instructions

1. For the same season, convert each category index in idx to an 8-hour distance using a log-normal model.
2. Use rlnorm() to draw N distances.
3. Use meanlog[idx] and sdlog[idx] so that each ignition uses the parameters for its own category.
4. Store the distances in a vector **d**.
5. Inspect the first few values and a summary of d.

□ Goal. *Write your own sentence(s) goal for turning category indices into distances for one season.*

```
# □ Test: simulate distances for one season
____ # store d, use rlnorm() with n = N, meanlog = meanlog[idx], sdlog = sdlog[idx]
```

```
# □ Verify: examine distances

____ # show head()
____ # summarize d
```

□ Comment (TODO). *Describe what you notice about the distribution of distances across all categories.*

4.7 Task 7

Count fast runs for one season

Task instructions

1. Using the vector d, mark which ignitions are "fast-growing" using the condition d >= k.
2. Store the logical vector in **fast_flags_one**.
3. Use sum() on fast_flags_one to count fast ignitions and store the result as **fast_count_one**.
4. Compute mean() of fast_flags_one to get the fraction of fast ignitions.

□ Goal. *Write your own sentence(s) goal for counting fast-growing ignitions.*

```
# □ Test: count fast runs in one season
____ # create fast_flags_one as a logical vector using d >= k
____ # compute fast_count_one using sum() of fast_flags_one
```

```
# □ Verify: inspect count and proportion
____ # display fast_count_one
____ # optionally compute mean() of fast_flags_one to get proportion
```

□ Comment (TODO). Comment on how many fast runs you saw in this particular simulated season.

4.8 Task 8

Bundle one-season logic in a single block

Braces { ... } group several lines together so they act as one expression and return the final result.

Task instructions

1. Combine the one-season computation into a single block that:

- samples idx for one season,
- simulates distances d,
- returns the count of fast ignitions.

2. Wrap the block in braces {} and assign the result to **fast_count_one**.
3. Run the block and confirm it returns a single number (the fast count).

□ Goal. *Write your own sentence(s) goal for packaging the one-season simulation into one code block.*

```
# □ Test: bundled one-season simulation returning a single fast count
____ <- { #store vector and start block
____ # sample idx for one season
____ # simulate distances d using meanlog, sdlog with brackets and idx
____ # return the count of d values that are greater than or equal to k
} #end block
```

```
# □ Verify: confirm the block works
____ # display fast_count_one
```

□ Comment (TODO). *Explain why having this one-season block is useful when you move to many seasons.*

Task 8 might have been challenging! Consider other kinds of memos: □ Reflect; □ Growth mindset; □ Cite resources; □ Code trials; □ Record what you tried.

4.9 Task 9

Use replicate() to simulate many seasons

Task instructions

1. Use replicate() to repeat your one-season block S times.
2. Store the resulting vector of counts in **counts_fast_mix**.
3. Compute the expected number of fast runs per season as the mean of counts_fast_mix.
4. Estimate the probability that a season has at least 50 fast-growing events by computing the fraction of seasons where the count is ≥ 50.

□ Goal. *Write your own sentence(s) goal for simulating many seasons and estimating the chance of at least 50 fast fires.*

```
# □ Test: many-season simulation with replicate()
set.seed(2025)
____ # store resulting vector, use replicate()
____ # with n = number of seasons
____ # and expr = { ... one-season block ... } as before
)
```

```
# □ Test: Answer Question
# summarize seasonal fast counts and estimate probabilities
_____ # inspect summary() of counts_fast_mix
_____ # compute mean() for expected fast runs
_____ # compute mean() for Monte Carlo estimate of P(T >= 50)
```

□ Comment (TODO). *Interpret both the expected fast count and the estimated probability of at least 50 fast fires in words.*

Task 9 might have been challenging! Consider other kinds of memos: □ *Reflect;* □ *Growth mindset;* □ *Cite resources;* □ *Code trials;* □ *Record what you tried.*

4.10 Task 10

Visualize the distribution of seasonal fast counts

Task instructions

1. Use ggplot2::geom_histogram() (or base hist()) to plot a histogram of counts_fast_mix.
2. If you use ggplot2, you could first build a small data frame with one column (fast_count).
3. Clearly label the axes and choose a reasonable bin width.
4. Draw a vertical line at 50 to mark the threshold and visually highlight seasons with ≥ 50 fast fires.
5. Relate the shape of the histogram to your numeric summaries from Task 9.

□ Goal. *Write your own sentence(s) goal for plotting the distribution of fast fires per season.*

```
# □ Test: histogram of seasonal fast counts

_____ # load ggplot2 if you choose to use it
_____ # create a data frame with a column fast_count = counts_fast_mix

_____ # build a histogram of fast_count (e.g., ggplot(...) + geom_histogram(...))
_____ # Add a vertical line at the threshold of 50 fast fires
```

```
# □ Verify: inspect histogram
_____ # display the histogram
```

□ Comment (TODO). *In one sentence, explain how the histogram supports (or challenges) your numeric estimate of the probability of ≥ 50 fast fires.*

5. Save and Upload

Submit your (1) completed assignment notebook (R Markdown/Quarto file) (2) rendered HTML and (3) environment (RData) to Canvas.

6. Today you practiced:

- Importing raw data and auditing structure, missingness, and ranges.
- Building quantile-based hazard categories with mutate(), case_when(), and ordered factors.
- Summarizing area and probabilities by category with group_by(), summarize(), and inline calculations.
- Working with lognormal models using plnorm(), qlnorm(), dlnorm(), and rlnorm() to answer distance and tail-probability questions.
- Designing and running Monte Carlo simulations with sample.int(), vectorized parameters, and replicate().
- Visualizing simulated outcomes and checking model behavior with histograms.
- Keeping a reproducible, memo-rich Rmd/Qmd workflow that documents assumptions, troubleshooting, and AI use.

 Keep pushing further. Clean imports, clear summaries, probability models, and simulations are exactly what you'll need for larger projects and future R courses.